Laporan Praktikum Algoritma & Pemrograman

Semester Genap 2025/2026

SAYA MENYATAKAN BAHWA LAPORAN PRAKTIKUM INI SAYA BUAT DENGAN USAHA SENDIRI TANPA MENGGUNAKAN BANTUAN ORANG LAIN. SEMUA MATERI YANG SAYA AMBIL DARI SUMBER LAIN SUDAH SAYA CANTUMKAN SUMBERNYA DAN TELAH SAYA TULIS ULANG DENGAN BAHASA SAYA SENDIRI.

SAYA SANGGUP MENERIMA SANKSI JIKA MELAKUKAN KEGIATAN PLAGIASI, TERMASUK SANKSI TIDAK LULUS MATA KULIAH INI.

NIM	**71251230**
Nama Lengkap	**Okky Alexander**
Minggu ke / Materi	**05 / Modular Programming**

PROGRAM STUDI INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS KRISTEN DUTA WACANA
YOGYAKARTA
2026

MATERI FUNGSI, ARGUMENT DAN PARAMETER

Dalam pemrograman, kita sering menggunakan fungsi tanpa menyadarinya. Contoh sederhananya adalah saat kita menggunakan input() untuk menerima data dari pengguna dan print() untuk menampilkan teks ke layar. Keduanya adalah built-in functions atau fungsi bawaan yang sudah disediakan oleh Python. Secara umum, fungsi adalah sekumpulan perintah yang dikelompokkan menjadi satu kesatuan untuk menjalankan tugas spesifik. Fungsi bersifat reusable, artinya dapat dipanggil berkali-kali tanpa harus menulis ulang kodenya. Konsep ini berkaitan erat dengan Modular Programming. Bayangkan sebuah program besar yang kompleks; untuk mempermudah pengerjaannya, kita membagi program tersebut menjadi bagian-bagian kecil (modul) yang memiliki peran khusus.

fungsi dibedakan menjadi dua:

1. Fungsi Bawaan (Built-in): Fungsi standar Python (seperti len(), type(), print()).
2. Fungsi Buatan Sendiri (User-defined): Fungsi yang didefinisikan sendiri oleh programmer sesuai kebutuhan.

Struktur dan Cara Kerja Fungsi

contoh fungsi tambah() berikut:

```
def tambah(a, b):
    hasil = a + b
    return hasil

c = tambah(10, 5)
print(c)
```

Gambar 5.1: fungsi tambah()

Komponen Penting:

- def: Keyword untuk mulai mendefinisikan fungsi.
- Parameter (a, b): Variabel penampung input yang dibutuhkan fungsi.
- Indentasi: Seluruh isi fungsi harus menjorok ke dalam (1 tab/4 spasi) untuk menandakan blok kode fungsi tersebut.
- return: Keyword untuk mengirimkan hasil olahan fungsi kembali ke pemanggilnya.

Alur Eksekusi Program:

1. Python membaca definisi fungsi (namun tidak langsung menjalankannya).
2. Saat baris c = tambah(10, 5) dipanggil, program akan "melompat" masuk ke dalam fungsi.
3. Nilai 10 masuk ke parameter a, dan 5 ke b.
4. Hasil penjumlahan dikalkulasi dan dikirim balik melalui return.
5. Program kembali ke baris pemanggilan, mengisi variabel c dengan nilai 15, lalu mencetaknya.

Urutan penulisan sangat berpengaruh. Sebuah fungsi harus didefinisikan terlebih dahulu sebelum dipanggil. Jika Anda memanggil fungsi yang kodenya baru ditulis di bawah baris pemanggilan, Python akan memberikan pesan error.

MATERI RETURN VALUE

Dalam Python, fungsi dapat dibedakan menjadi dua jenis utama berdasarkan apakah fungsi tersebut memberikan hasil (output) yang bisa diolah kembali atau tidak.

1. Fungsi Tanpa Nilai Kembalian (Void Function)

Fungsi jenis ini hanya bertugas menjalankan serangkaian instruksi (seperti mencetak teks) tanpa mengirimkan data balik ke pemanggilnya. Secara teknis, jika kita mencoba mengambil nilai dari fungsi ini, Python akan memberikan nilai None.

Contoh:

```
def sapa_dua_kali(pesan):
    print(pesan)
    print(pesan)

sapa_dua_kali("Halo!")
```

Gambar 5.2: fungsi sapa_dua_kali.

Pada contoh di atas, fungsi sapa_dua_kali hanya menampilkan teks di layar. Kita tidak bisa menyimpan hasil fungsi tersebut ke dalam variabel untuk perhitungan matematika atau proses lainnya.

2. Fungsi dengan Nilai Kembalian (Return Function)

Fungsi ini menggunakan kata kunci return untuk mengeluarkan hasil pemrosesan agar dapat digunakan pada tahap program selanjutnya.

Kegunaan utama return:

- Mengirim data: Mengeluarkan hasil akhir dari dalam fungsi ke luar.
- Menghentikan fungsi: Segera mengakhiri eksekusi fungsi begitu baris return tercapai.

Contoh Implementasi (Menghitung Rata-Rata): Fungsi yang mengembalikan nilai sangat berguna jika hasilnya ingin kita olah lagi, misalnya menjumlahkan tiga angka untuk kemudian dicari rata-ratanya:

```
def tambah(a, b, c):
    hasil = a + b + c
    return hasil

nilai1, nilai2, nilai3 = 70, 85, 55
rata_rata = tambah(nilai1, nilai2, nilai3) / 3
print(rata_rata)
```

Gambar 5.3: Fungsi Menghitung Tambah dan Rata-Rata.

Alur Program:

1. Definisi: Python mencatat adanya fungsi tambah, namun belum mengeksekusinya.
2. Inisialisasi: Variabel nilai1, nilai2, dan nilai3 diisi.
3. Pemanggilan: Saat baris rata_rata dibaca, program melompat ke fungsi tambah dengan membawa data (70, 85, 55).
4. Proses & Return: Fungsi menghitung total (210) dan mengirimnya kembali ke baris pemanggil.
5. Finalisasi: Nilai 210 yang diterima kemudian dibagi 3, disimpan ke variabel rata_rata, dan ditampilkan ke layar.

MATERI OPTIONAL ARGUMENT DAN NAMED ARGUMENT

Python memberikan keleluasaan bagi kita untuk mengatur bagaimana data dimasukkan ke dalam fungsi. Dua fitur utamanya adalah Optional Argument (nilai bawaan) dan Named Argument (pemanggilan berdasarkan nama).

1. Optional Argument (Parameter Opsional)

Terkadang, ada data yang tidak selalu perlu diinput oleh pengguna karena sudah memiliki nilai standar. Di sinilah kita menggunakan Default Value. Jika kita tidak memberikan nilai saat fungsi dipanggil, Python akan menggunakan nilai bawaan tersebut.

```python
def hitung_belanja(total, diskon=0):
    bayar = total - (total * diskon / 100)
    return bayar
print(hitung_belanja(100000))
print(hitung_belanja(100000, 10))
```

Gambar 5.4: Fungsi Hitung Diskon Belanja.

Pada fungsi di atas, diskon adalah parameter opsional. Kita bisa memanggilnya dengan satu angka saja (hanya total) atau dua angka sekaligus.

2. Named Argument (Keyword Argument)

Secara standar, Python memasukkan data ke fungsi berdasarkan urutan posisinya (Positional Arguments). Namun, kita juga bisa memanggil fungsi dengan menyebutkan nama parameternya secara eksplisit.

Keuntungan menggunakan Named Argument:

- Urutan tidak jadi masalah: Anda bisa memasukkan data secara acak selama namanya benar.
- Kode lebih mudah dibaca: Orang lain tahu angka tersebut diperuntukkan bagi variabel apa.

```python
def cetak_data(a, b, c):
    print(f"A: {a}, B: {b}, C: {c}")
cetak_data(10, 20, 30)
cetak_data(b=20, c=30, a=10)
```

Gambar 5.5 Fungsi Hitung Diskon Belanja.

Walaupun urutan pada pemanggilan kedua diacak menjadi b, c, a, Python tetap pintar menempatkan nilai 10 ke dalam a karena kita sudah menyebutkan namanya dengan jelas.

MATERI ANONYMOUS FUNCTION (LAMBDA)

Dalam Python, Anonymous Function atau fungsi anonim adalah fungsi yang didefinisikan tanpa menggunakan nama. Berbeda dengan fungsi standar yang menggunakan def, fungsi ini bersifat ringkas dan biasanya digunakan untuk tugas-tugas sederhana yang hanya memerlukan satu baris kode.

Fitur ini diadaptasi dari bahasa pemrograman fungsional (seperti Lisp atau Haskell) dan diimplementasikan dalam Python menggunakan kata kunci lambda.

Perbandingan Struktur: def vs lambda

Mari kita lihat bagaimana fungsi penjumlahan sederhana ditulis dalam dua cara berbeda:

1. Menggunakan Fungsi Standar (def):

```
def tambah(a, b):
    return a + b

print(tambah(10, 20))
```

Gambar 5.6: Fungsi Standar Menghitung Tambah (def):.

2. Menggunakan Fungsi Lambda:

```
tambah = lambda a, b: a + b

print(tambah(10, 20))
```

Gambar 5.7: Fungsi Lambda Menghitung Tambah (def):.

Anatomi Fungsi Lambda

Setiap fungsi lambda di Python selalu terdiri dari tiga bagian utama:

1. Keyword lambda: Kata kunci wajib untuk memulai pendefinisian fungsi anonim.
2. Bound Variable: Argumen atau input yang dibutuhkan oleh fungsi (dalam contoh di atas adalah a dan b).
3. Body: Berisi ekspresi atau pernyataan matematika tunggal. Hasil dari ekspresi ini akan langsung dikembalikan (return) secara otomatis tanpa perlu menulis kata kunci return.

Mengapa Menggunakan Lambda?

Meskipun bukan fitur utama dalam Python, lambda sangat berguna saat Anda membutuhkan fungsi kecil untuk sementara waktu, misalnya saat bekerja dengan fungsi tingkat tinggi seperti map(), filter(), atau sort() yang membutuhkan fungsi lain sebagai argumennya.

Source Github : https://github.com/tuangkeman/71251230_Okky.git

LATIHAN 5.1

```
def cek_angka(a,b,c):
    if a != b and b != c and c != a:
        if (a + b) == c or (b + c) == a or (a + c) == b:
            return True
        else:
            return False
    else:
        return False
```

Gambar 5.7: Program Fungsi Cek Angka.

Fungsi cek_angka(a, b, c) dirancang untuk memvalidasi dua kriteria logis pada tiga parameter masukan sekaligus. Pertama, fungsi melakukan filter awal menggunakan operator perbandingan != dan logika and untuk memastikan bahwa ketiga angka tersebut memiliki nilai yang unik atau berbeda satu sama lain. Jika syarat keunikan ini terpenuhi, program akan masuk ke percabangan kedua yang memeriksa hubungan matematis antar angka, yaitu apakah penjumlahan dari dua angka tertentu akan menghasilkan nilai yang sama dengan angka yang tersisa. Pemeriksaan ini mencakup tiga kemungkinan kombinasi menggunakan operator or, sehingga jika salah satu kombinasi penjumlahan terbukti benar, fungsi akan memberikan nilai balik True. Namun, jika salah satu dari kedua syarat utama tersebut tidak terpenuhi—baik karena ada angka yang kembar maupun tidak adanya kecocokan hasil penjumlahan maka fungsi secara otomatis akan mengembalikan nilai False melalui blok else.

LATIHAN 5.2

```python
def cek_digit_belakang(a, b, c):
    x = a % 10
    z = b % 10
    v = c % 10
    if x == z or z == v or v == x:
        return True
    else:
        return False

try:
    a = int(input("Masukkan Input pertama: "))
    b = int(input("Masukkan Input kedua: "))
    c = int(input("Masukkan Input ketiga: "))

    hasil = cek_digit_belakang(a, b, c)
    print(f"Output = {hasil}")

except ValueError:
    print("Mohon masukkan angka bulat yang valid.")
```

Gambar 5.7: Program Fungsi Cek Digit Belakang.

Kode tersebut berfungsi untuk mengecek apakah ada minimal dua angka yang memiliki digit terakhir yang sama dari tiga input pengguna. Di dalam fungsi cek_digit_belakang, setiap angka diambil satuannya menggunakan operasi modulo 10 (% 10), lalu dibandingkan satu sama lain menggunakan operator or; jika ada pasangan yang cocok, fungsi mengembalikan True.

Pada bagian utama, program menggunakan blok try-except untuk memastikan input yang dimasukkan benar-benar angka bulat. Jika input valid, fungsi akan dipanggil dan hasilnya dicetak, namun jika input mengandung karakter non-angka, program akan menampilkan pesan peringatan tanpa mengalami *error* atau berhenti mendadak

LATIHAN 5.3

```
celtofag = lambda c: (9/5) * c + 32
celtorea = lambda c: 0.8 * c
o=100
print(f"Input C = {o}. Output R = {celtofag(o)}")
x=80
print(f"Input C = {x}. Output R = {celtorea(x)}")
l=0
print(f"Input C = {l}. Output R = {celtofag(l)}")
```

Gambar 5.7: Program Fungsi Celcius Ke Fahrenheit/Reamur.

Kode ini mengimplementasikan fungsi anonim atau lambda function untuk melakukan konversi suhu dari satuan Celcius ke Fahrenheit dan Reamur secara ringkas. Dengan menggunakan keyword lambda, rumus matematika seperti $F = \frac{9}{5}C + 32$ dan $R = 0.8C$ dapat dituliskan dalam satu baris saja tanpa memerlukan blok def yang panjang.

Variabel celtofag dan celtorea menyimpan logika fungsi tersebut, sehingga ketika variabel o, x, dan l dimasukkan sebagai argumen, program langsung melakukan kalkulasi dan mengembalikan hasilnya. Penggunaan f-string pada fungsi print mempermudah penyajian data secara informatif, yang menunjukkan bahwa input 100°C menghasilkan 212°F, 80°C menghasilkan 64°R, dan titik beku 0°C setara dengan 32°F.